

1 Introduction

Humans observe the world through their eyes.

2 Point and vector

In Veritas, we will strictly distinguish between points and vectors. Points in space can be described using tuples $(x, y, z, \dots)$. A vector can be described as a directed line segment between two points.

```
1 struct P{}; struct V{};
2 template<typename T, int N> struct vec_type;
3 template<typename T> struct vec_type<T, 2>{T x, y;};
4 template<typename T> struct vec_type<T, 3>{T x, y, z;};
5 template<typename T, typename tag, int N>
6 struct vec: vec_type<T, N>{
7     HD vec();
8     HD vec(T x, T y) requires (N==2);
9     HD vec(T x, T y, T z) requires (N==3);
10    template<typename tg> HD vec(const vec<T, tg, N>&v);
11    HD T&operator[](int x);
12    HD const T&operator[](int x) const;
13    #define VEC(op)\
14    template<typename U>\
15    HD auto operator op (U s) const{vec<decltype(this->x op s), tag, N>ans; int x;
16        for(x=0; x<N; x++) ans[x]=(&this->x)[x] op s; return ans;}
17    VEC(*); VEC(/);
18    #undef VEC
19    #define VEC_OP(op, tag1, tag2)\
20    HD vec<T, tag2, N> operator op (const vec<T, tag1, N>&v) const{vec<T, tag2, N>ans;
21        int x; for(x=0; x<N; x++) ans[x]=(&this->x)[x] op v[x]; return ans;}
22    VEC_OP(+, tag, tag); VEC_OP(-, V, V);
23    #undef VEC_OP
24    HD vec<T, tag, N> operator -() const;
25    HD bool operator==(const vec<T, tag, N>&a) const;
26    HD bool same(const vec<T, tag, N>&a, const vec<T, tag, N>&b) const;
27 };
28 Basic point and vector operations satisfy the following rules:
29 \[point+point=point\]
30 \section{Ray}
31 A ray of light is defined by us as directed line segment. It consists of a
32 starting point  $o$  and an ending point  $d$ .
33 so the ray can be written by  $r(t)=o+td$ . where  $t$  is the distance along the
34 direct of the ray.
35 \begin{lstlisting}[language=C++]
36 template<typename T>
37 struct ray{
38     vec3<T, P>o;
39     vec3<T, V>d;
40     T t, tmn;
41     mutable T tmx;
42     T time;
```

```

39     HD ray();
40     HD ray(const vec3<T,P>&o, const vec3<T,V>&d, T t, T tmx, T tmn, T time);
41     HD T operator()(T t);
42 };

```

also, tmx was used for close-range pruning and initially it is infinite. During the ray traversal, we need to continuously update tmx to smaller t . tmn is set as the starting the light rays, primarily to prevent light rays from self-intersecting. And $time$ is used to discribe dynamic blur, which we will explain detail later.

3 Camera

Camera is one of the important part in render. It bears the important responsibility of emitting light. We cannot define the generation of photons by the sun. To facilitate the addition of a variety of camera, we designed it as follows CRTP.

```

1  template<typename tp, typename T> class C{
2  template<typename samp>
3  public: void Ray(Pixel<T, samp>&a, Path<T>*b){static_cast<const tp*>(this)->gt(a,b)
        ;} int w=500, h=500;};

```

3.0.1 Depth of filed camera

In real life, a camera can only focus precisely on an object at a precise distance. This phenomenon lead means that objects on the focal plane are absolutely sharp, while those away from the focal plane become blurry; the larger the circle of confusion, the circle of confusion, the blurrier the object appears. In order to get maximum DOF, we will introduce the concept of hyperfocal distances It will be given:

$$H = f + \frac{f^2}{Nc}$$

where f is the focal length, N is the f-number, c is the confusion of circle. If the lens is focused at the hyperfocal distance, then the range from half of that distance to infinity will be within the depth of field. The depth of field formula will be given:

$$D_N = \frac{sf^2}{f^2 + cN(s - f)}$$

$$D_F = \frac{sf^2}{f^2 - cN(s - f)}$$

$$D_{DOF} = D_F - D_N$$

but in this, we will use some easy method. Suppose we look at w and the ray is $r(t) = o + td$, so:

$$r' = o + \frac{f}{d \cdot w}d$$

```

1  //dof
2  template<typename T>
3  C1<T>::C1(Pose<T>pose, T fov, int w, int h, T dis, T rds): pos(pose), dis(dis), rds(rds)
4  {
    this->w=w; this->h=h; hf_h=tan(fov/2*3.1415926535/180.0); hf_w=T(w)/T(h)*hf_h; }

```

```

5  template<typename T>
6  template<typename sp>
7  HD void C1<T>::gt(Pixel<T,sp>&a,Path<T>*b) const{
8      T u,v;a->sample.imp2(u,v);T fx=a.px+u,fy=a.py+v;
9      T ndcx=(T(2)*fx)/this->w-T(1),ndcy=T(1)-(T(2)*fy)/this->h;
10     vec3<T,V>dir=nor(pos.f+pos.r*ndcx+pos.u*ndcy);
11     if(rds>T(0)){
12         T dx,dy;sp(&dx,&dy,b->sample);
13         T lx=dx*rds,ly=dy*rds;vec3<T,V>pt=pos.p+pos.r*lx+pos.u*ly;
14         vec3<T,V>ed=pos.p+dir*(dis/dot(pos.f,dir));
15         b.r->o=pt;b.r->d=nor(ed-pt);
16     }else{b->sample.dim(2);b.r->o=pos.p,b.r->d=dir};
17     b.r->t=a.t;
18 }
19 template<typename T>
20 HD void C1<T>::sp(T*dx,T*dy,sampler<T>&s) const{
21     T u,v;s.imp2(u,v);
22     T phi,r;if(!u&&!v){*dx=*dy=0;return;}
23     const T pi=3.14159265358979323846;
24     if(u>v&&u>-v)r=u,phi=3.1415926535/T(4)*u/v;
25     if(u<v&&u>-v)r=v,phi=3.1415926535*(T(2)-u/v);
26     if(u<v&&u<-v)r=-u,phi=3.1415926535/T(4)*(T(4)+u/v);if(u>v&&u<-v)r=-v,phi
        =3.1415926535/180.0/T(4)*(T(6)-u/v);
27     *dx=r*cos(phi);*dy=r*sin(phi);}

```